Reference Architecture Brief: API Management

30 October 2025 - ID G00839499 - 28 min read

By: Steve Deng

Initiatives:Software Architecture and Integration for Technical Professionals; Adopt Modern Architectures and Technologies; Build and Deliver Software to Drive Business Results

APIs and effective API management are essential for modern applications, AI and integrations. This research provides a comprehensive reference architecture for well-managed APIs that will help you design, implement and optimize API ecosystems that drive innovation and business agility.

Architecture Brief

APIs and API management (APIM) are foundational practices in modern application architecture. They play a critical role in environments such as microservices architectures (MSA), agile web applications, cloud-native applications, and application integration. Building AI-enabled applications and AI agents also requires robust API management.

To maximize the potential of APIs, you need robust tools and processes to manage the entire API life cycle. This includes planning and initial design, building, testing, deployment, versioning, and eventual retirement.

Essential components for effective API management include:

- An API control plane, which offers analytics and life cycle management tools to monitor and govern API usage.
- An API portal, which enables developers to discover and adopt available APIs.
- API gateways, which provide runtime management and security.

A core capability for runtime API management is the API gateway. API gateways secure north-south API traffic at the network, enterprise, cloud, and data center edges. They also mediate east-west traffic within the enterprise, facilitating communication between applications and services.

This research note is restricted to the personal use of .

This reference architecture presents a structured framework for designing and implementing API management across diverse enterprise use cases. While HTTP/HTTPS remains the dominant protocol for most API management scenarios, support for additional protocols like gRPC, GraphQL, SOAP, websockets, and event-driven APIs is increasingly common to address modern enterprise environments. Practices and technical capabilities for other API types — such as LLM-as-a-service APIs, MCP, and A2A — are rapidly evolving. These emerging scenarios introduce new components and capabilities as natural extensions to traditional APIM architecture.

Architecture Use Cases

API management architecture commonly supports the following use cases:

- **Offering external-facing APIs:** In this scenario, you are exposing services and data to external developers via public APIs or partner APIs. Commercializing APIs that deliver access to valuable data or business capabilities can open up new revenue and partnership channels.
- **Supporting MSA:** In this scenario, you are using MSA principles to build applications, and they need to manage and secure service-to-service and app-to-service communication.
- **Implementing API-based integration:** In this scenario, you use APIs to integrate internal and external applications, services and data. These APIs need to be secured, managed and monitored.
- **Productizing APIs:** This use case involves grouping related APIs into scalable, user-friendly products that address specific business needs. When shared beyond their original domain, APIs should be managed as products with clear technical and nontechnical contracts. This approach builds trust, increases adoption, and enables scalable innovation across the organization.
- **Managing APIs in hybrid/multicloud environments:** In this scenario, you need to manage, secure and monitor APIs consistently across hybrid and multicloud environments.
- **Managing third-party APIs:** In this scenario, you consume third-party APIs (e.g., mapping APIs, payment APIs, SaaS application APIs, LLM APIs, or other business partner APIs). You want to control and monitor the usage of those APIs, either to track and limit costs or to safeguard sensitive data.

This research note is restricted to the personal use of .

- **Managing LLM access:** You are using a custom-configured or purpose-built API gateway (often called an AI/LLM gateway) to secure, mediate, manage, and observe AI service consumption across multiple LLM providers or infrastructure. This is a special version of the Managing third-party APIs use case.
- **Enabling AI integration:** Traditional API gateways, along with purpose-built AI-ready gateways that support emerging AI-centric protocols such as MCP and A2A, facilitate and simplify AI tool use and multiagent communication, as well as integration with the surrounding digital and physical environments.

Architecture Diagram

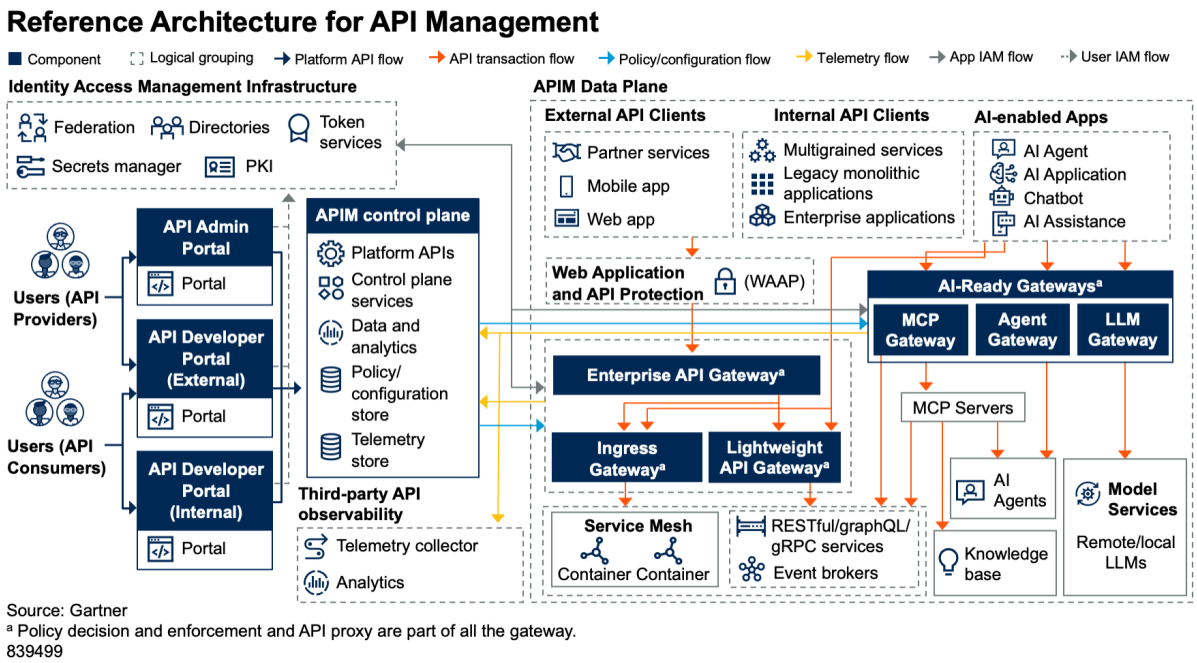
Figure 1 shows API management architecture in a dual-plane configuration. These components must be deployed and integrated into your operational environment. The figure also shows the relationships and interactions among these components, using colored arrows to denote:

- Policy/configuration flow
- API transaction flow
- Telemetry flow
- Platform API flow
- App identity and access management (IAM) flow
- User IAM flow for portal access

Note that this is an abstract reference architecture. You will need to customize it to your specific technologies and circumstances.

This research note is restricted to the personal use of .

Figure 1: Reference Architecture for API Management



Click to Download an Editable Version of This Figure

The reference architecture shows six types of connectivity:

- **Platform API flow:** The integration between the API admin portal, API developer portals or integrated development environments (IDEs), and other APIM components via platform APIs provided by the control plane.
- **API transaction flow:** The flow of messages (e.g., requests, responses, commands or events) between API consumers and the exposed service implementations. This flow is mediated by one or more API gateways and optionally filtered by web application and API protection (WAAP) capabilities.
- **Policy/configuration flow:** The flow of policy or configuration updates from the control plane to the API gateway instances making up the APIM data plane.
- **Telemetry flow:** The asynchronous flow of observability and API usage metrics from the API gateways to the control plane and telemetry collectors.

This research note is restricted to the personal use of .

- **App IAM flow:** The flow of access control requests from the control plane and data plane to the enterprise IAM to utilize authentication, authorization, public-key infrastructure (PKI), directories and secrets management services. The identities being authenticated here may include:
  - Application users (of the apps that employ the API)
  - Application/service accounts for apps that don't delegate user identity checks
- **User IAM flow:** The flow of authentication/authorization (AuthN/AuthZ) requests from the API admin portal, command line interface (CLI) and API developer portals to the enterprise IAM to authorize provider- and consumer-user access to the platform. Single sign-on (SSO) is also supported using this flow. Note that different identity providers and tools may be used for the consumer identities versus the provider/admin identities.

The core components of an APIM platform are:

- **APIM data plane:** Components (mainly API gateways) responsible for mediating APIs implemented by the API providers and making them securely accessible by the API consumers. The four primary types of API gateways in the APIM data plane are:

- Enterprise API gateway: A component typically located on the network perimeter to secure inbound and outbound API traffic
- Lightweight API gateway: A component designed to be distributed and deployed close to individual services
- Ingress gateway: A component that functions as the network communications gatekeeper for a container orchestration cluster or service mesh
- AI-ready gateways: a class of API gateways that is purpose-built or custom-configured to mediate AI agent interactions with AI services implementations. This class of gateways includes AI/LLM/model gateways, MCP gateways, and agent gateways.

This research note is restricted to the personal use of .

- APIM control plane: The software and services that manage API gateway configurations, API proxy definitions and APIM policies, including security, traffic management, rate limiting, monitoring and caching. The APIM control plane also provides platform APIs to integrate with API developer portals, API admin operations and data plane interfaces, among others.
- API admin portal: The UI component (either a web portal or a CLI) for administrators to manage API gateways and global policies, and for API providers to manage API proxy definitions and application-specific policies.
- API developer portal: The UI component (typically a web portal) for API consumer developers to discover, test-drive and request access to APIs published in the catalog.

The following are external components that frequently integrate with APIM architecture. These components are covered in other Gartner Reference Architecture Briefs and research:

- **Web application and API protection:** WAAP solutions support security controls to help protect the back-end service environment from malicious attacks or accidental misuse. These solutions encompass a broad set of protection capabilities and features for web-facing applications and services, including:
  - Bot detection to discover unauthorized automation and attacks
  - Distributed denial of service (DDoS) protection to defend against brute-force saturation of service capacity
  - Web application firewalls (WAFs) to protect against web-technology-based attacks, such as malformed requests, script injection and cross-site request forgery (CSRF)

This research note is restricted to the personal use of .

- **Identity access management infrastructure:** This is a collection of IAM tools that are architected to enable identity-first security. It typically includes the following components:
  - Access management
  - Identity federation
  - Token services
  - Application directory
  - PKI
  - Secrets manager
- **Thirty-party API observability and security audit:** This is an optional group of API observability components that collect and store usage metrics and operational telemetry from the APIM control plane or API gateways. The telemetry collected can support custom analytics/reporting (e.g., for digital experience monitoring and monetization). With additional integration, the telemetry can also be leveraged by some security tools for security audit purposes. Software observability components typically include (see Reference Architecture Brief: Software Observability):
  - Telemetry collector
  - Telemetry data store
  - Telemetry analytics
  - Telemetry visualization

Architecture Capabilities and Components

The following subsections provide an overview of each component of the reference architecture.

This research note is restricted to the personal use of .

Enterprise API Gateway

An enterprise API gateway is typically located on the network perimeter to secure inbound and outbound API traffic. It is best-suited for external-facing, productized and published APIs. These gateways are often centrally managed by the organization and may be part of a broader APIM solution that includes API life cycle management and an API developer portal.

Enterprise API gateways commonly support:

- Access management (authentication and authorization)
- Traffic management (quotas and distributed rate limiting)
- Request routing
- Request and response payload validation
- Payload filtering or transformation
- Caching
- Coarse-grained security policy enforcement

This research note is restricted to the personal use of .

Example Technologies

- Amazon API Gateway
- APISix
- Axway Amplify API Management — API Gateway
- Google Cloud Apigee and Apigee hybrid API gateway
- Google Cloud API gateway
- Gravitee.io API Management
- IBM API Connect API gateway
- Kong Gateway
- Microsoft Azure APIM API Gateway
- MuleSoft Anypoint Platform API Gateway
- WSO2 API Gateway

Key Characteristics

An enterprise API gateway implementation has the following general characteristics:

- It must secure APIs at the edge of the enterprise, ensuring consistency in the overall API security strategy. Typically, only coarse-grained policies are enforced here.
- It should make APIs available to external audiences, either as public APIs, restricted private APIs, or partner-specific APIs.
- It should publish internally hosted APIs or third-party APIs to internal applications or services.
- It should manage access to APIs as products. These APIs are productized and published in a developer portal or API marketplace.
- It should support business value reporting and support direct or indirect monetization, aiming to contribute value to the business.

An enterprise API gateway implementation has the following technical characteristics:

This research note is restricted to the personal use of .

- It must integrate with WAAP and enterprise IAM infrastructure for all external-facing north-south mediation
- It should support flexible deployment models (for example, SaaS and/or a customer self-managed, on-premises, etc.).
- It may require a multicomponent runtime, which can increase operational overhead in customer self-managed environments.

Related Architecture Research

- Decision Point for Selecting the Right API Mediation Technology
- Comparing Architectures for Hybrid and Multicloud API Management
- How to Evaluate API Management Solutions
- How to Deliver Sustainable APIs

Lightweight API Gateway

A lightweight API gateway, aka microgateway, is designed to be distributed and deployed close to individual services. Unlike enterprise API gateways, which are often centrally managed as shared middleware, lightweight gateways can be individually deployed, configured and managed by a development team to meet the needs of the application.

Lightweight API gateways commonly support:

- Access management (authentication and authorization)
- Traffic management (quotas and distributed rate limiting)
- Request routing
- Request and response payload validation
- Payload filtering or transformation
- Caching
- Basic security enforcement
- East-west APIs and services communication.

This research note is restricted to the personal use of .

Example Technologies

- Apache APISIX
- Contour
- CNCF kgateway
- Envoy Gateway
- Kong Gateway
- Microsoft Azure self-hosted API gateway
- MuleSoft Anypoint Flex Gateway
- Solo.io Gloo Gateway
- Spring Cloud Gateway
- Tyk Gateway
- WSO2 API Platform for Kubernetes

Key Characteristics

A lightweight API gateway implementation has the following general characteristics:

- It should be optimized as a self-contained turnkey solution for customers to deploy and operate in private clouds or in on-premises data centers.
- It should be configurable to keep its resource footprint to a minimum.
- It should allow organizations to confine internal API traffic within their trusted network for security and compliance reasons by placing the lightweight API gateway in close proximity to API providers and API consumers.
- It should allow certain lightweight gateway instances to enforce domain-specific, fine-grained API policies as a way to implement a defense-in-depth strategy.
- It can be deployed as part of the organization's DevOps process. Configuration and control can be delegated to the development team.

A lightweight API gateway implementation has the following technical characteristics:

This research note is restricted to the personal use of .

- Its lightweight runtime footprint can be packaged in a single container or virtual machine (VM) image.
- It's built to be cloud-native. Recent implementations are built on the Envoy proxy.

Related Architecture Research

- Decision Point for Selecting the Right API Mediation Technology
- Comparing Architectures for Hybrid and Multicloud API Management
- How to Evaluate API Management Solutions
- How to Deliver Sustainable APIs

Ingress Gateway

In the context of this research, "ingress gateway" collectively refers to a class of components, often based on a lightweight gateway, that implement Kubernetes ingress functionality. Ingress gateways are differentiated from lightweight gateways only by their configuration model, which could be a Kubernetes Ingress controller, Kubernetes Gateway API, or a service mesh ingress gateway.

An ingress gateway is part of the container environment. It is a component that functions as the network communications gatekeeper for a container orchestration cluster or service mesh. All inbound traffic to services running inside the cluster must pass through the ingress gateway first. As a result, the ingress gateway becomes an ideal policy enforcement point (PEP) for protecting APIs implemented by the services deployed to the cluster or service mesh.

Example Technologies

Ingress gateways, which are implemented natively within Kubernetes environments, share many capabilities with lightweight API gateways. In fact, many lightweight API gateways can be readily extended to serve as ingress gateways. As a result, the following sample technologies demonstrate considerable overlap between ingress gateways and lightweight API gateways.

- Apache APISIX ingress controller
- Azure Application Gateway for Containers
- Contour

This research note is restricted to the personal use of .

- Envoy Gateway
- HashiCorp Consul API Gateway
- Kong Ingress Controller (KIC) for Kubernetes
- Mulesoft Anypoint Flex Gateway in connected mode
- Solo.io Gloo Gateway
- Traefik Kubernetes Ingress
- Tyk Gateway
- WSO2 API Platform for Kubernetes

Key Characteristics

An ingress gateway implementation has the following general characteristics:

- It must secure all external access to services running in Kubernetes or a service mesh.
- It must support a declarative configuration model to manage API mediation and service deployment.
- It should support the delegation of ingress routing rules to individual development teams. This capability gives development teams autonomy over the application-specific routing configurations while allowing the platform team to retain control over the entry point of the cluster.

An ingress gateway implementation has the following technical characteristics:

- It is often built on Envoy, with xDS as the most common data plane API.
- It must support Kubernetes CustomerResourceDefinition (CRD)-based configuration. It should support the Kubernetes Ingress Controller and/or Kubernetes Gateway API CRDs.
- It must minimize communication latency between gateway instances and the control plane. Options include using a local control plane or a local agent to bridge communication with remote control planes.

This research note is restricted to the personal use of .

- It may obtain policies from a remote APIM control plane via a local operator or directly from CRD.

Related Architecture Research

- Decision Point for Selecting the Right API Mediation Technology
- Comparing Architectures for Hybrid and Multicloud API Management
- How to Evaluate API Management Solutions
- How to Successfully Implement an API Management Platform
- How to Deliver Sustainable APIs

AI-Ready Gateways

AI-ready gateways are a class of purpose-built or custom-configured gateways tailored to secure, mediate and manage access to AI resources, services, and tools, including AI model services, MCP servers, APIs, AI-ready data, and AI agents. AI-ready gateways secure, route, manage, and observe AI interactions similar to how API gateways work for traditional API workloads. As a result, API management/API gateways vendors are offering specialized gateways for AI workloads, while others are building AI-ready gateway products from the ground up without lineage to an APIM platform.

Clients and consumers of AI-ready gateways are any AI-enabled applications and LLM-based agents.

There are different varieties of AI-ready gateways. They may be marketed as unified or separate products.

- **AI Gateway (or model/LLM router or proxy)** focuses on mediating access to LLM and other AI model services, with features such as model routing, semantic caching, AuthN/AuthZ, model budget optimization, model prompt/response guardrails, and observability. Some AI gateways may also include MCP support.
- **MCP gateway** is an emerging gateway component that mediates interactions between AI agents and external tools and resources using the model context protocol (MCP). It offers MCP-compliant features such as MCP server discovery, security, observability, and policy enforcement. A number of APIM vendors are introducing MCP gateways in their APIM product offerings.

This research note is restricted to the personal use of .

- **Agent Gateway** is another emerging gateway component focused on facilitating interactions and collaborations between AI agents using standard protocols, such as Google A2A.

Similar to API gateways, AI-ready gateways are offered as SaaS-managed services or as self-hosted software components. They may be managed by a unified control plane or independently configured in a declarative manner by developers and product teams.

Traditional API gateways are often optimized to handle high-throughput but short-lived API transactions. AI-centric transactions, on the other hand, (LLM access, agent interaction, and agent tool use) tend to be more long-lived, sometimes stateful, and have a long payload/context. As a result, AI-ready gateways are more CPU and memory-intensive to accommodate multiple large contexts across significantly more concurrent sessions.

Example Technologies

AI Gateways

- Axway AI Gateway
- Azure APIM AI Gateway
- Cloudflare AI gateway
- Envoy AI gateway
- IBM API Connect AI Gateway
- Gravitee AI gateway
- Kong AI Gateway
- Portkey
- Solo.io Gloo AI gateway
- Tetrate agent router services
- Vercel AI gateway

MCP Gateways:

- AWS Bedrock AgentCore Gateway

This research note is restricted to the personal use of .

- Azure APIM (with support for MCP Servers)
- Boomi MCP Bridge (preview)
- ContextForge MCP gateway
- CNCF kgateway
- Docker MCP Gateway
- Gravitee MCP gateway
- Truefoundary MCP gateway

Agent Gateways

- A2AGateway
- AgentGateway
- CNCF kgateway
- Gravitee Agent Mesh
- Tangletwo Agent Gateway

Key Characteristics

An AI gateway/LLM/router has these characteristics

- It presents a unified endpoint to access many LLMs and AI services.
- It routes requests to different LLM based on certain logic or criteria
- It load-balances requests across model instances to optimize for performance, latency, and costs.
- It protects LLM service endpoints and manages access via API keys or OAuth tokens.
- It enforces security and data privacy and content guardrails on LLM requests and responses.
- It enforces content moderation and PII masking and filtering rules

This research note is restricted to the personal use of .

- It provides logging, observability, and tracing capabilities on various LLM usage metrics
- It performs prompt validation, manages prompt templating and versioning
- It manages LLM token usage budget and cost optimization
- It provides semantic caching of frequent LLM completions
- It provides error handling, health monitoring, and retry logic on LLM services.

An MCP gateway has these emerging characteristics

- It presents a unified endpoint for AI agents to access external tools and services.
- It provides session/agent/task/process isolation and separation of concern with respect to tool selection, invocation, and context management.
- It load-balances and routes requests to the right MCP servers/tools.
- It can mediate MCP requests and route them to REST and other APIs.
- It secures access to MCP servers by providing consistent, fine-grained, and scoped access control.
- It often includes or provides tools to manage MCP server registration and discovery.
- It manages MCP tools token usage budget and cost optimization
- It provides logging, observability, and tracing capabilities between the AI agent and MCP servers.

Agent gateways are nascent and evolving technologies that facilitate multiagent interactions and collaboration. An agent gateway has these emerging characteristics:

- It supports one or more proprietary or open-standard agent-to-agent communication protocols, such as Google A2A or Agntcy ACP.
- It facilitates asynchronous communication between agents.
- It presents a unified endpoint for AI agents to access, engage, interact, and collaborate with other trusted agents.

This research note is restricted to the personal use of .

- It provides session/agent/task/process isolation and separation of concern with respect to agent selection, interaction, and context management.
- It includes or provides tools to facilitate agent registration, discovery and contract negotiation.
- It manages agent token usage budget, level of effort, quality-latency-cost optimization

Related Architecture Research

Choosing API Formats for AI Agents and Applications

Innovation Insight: AI Gateways

Innovation Insight: Model Context Protocol

Emerging Tech: Security Implications of Model Context Protocol

APIM Control Plane

The APIM control plane is typically provided as a part of an APIM suite. It is a set of core software and services that manage API gateway configurations, API proxy definitions, API management policies, and API monitoring and observability policies. The control plane hosts:

- APIs for dynamic configuration of the API gateways
- Platform APIs for communication with the API admin portal and API developer portals

The platform APIs can facilitate integration with external IDEs and DevOps processes. The control plane often contains one or more data stores or repositories to store API gateway configurations, API proxy definitions, API enforcement policies and API usage metrics. APIM control plane implementations also include an API catalog or registry, which can be accessed by API developer portals.

Runtime for the APIM control plane should be deployed and managed independently from the data plane. APIM control planes are typically offered as managed services from cloud providers or service providers. Only some APIM vendors offer self-hosted, customer-managed APIM control plane options.

This research note is restricted to the personal use of .

Example Technologies

- Amazon Web Services (AWS) Management Console
- Boomi API Control Plane
- APIwiz
- Axway Amplify Management Plane
- Envoy Gateway control plane
- Google Cloud Apigee (hybrid) management plane
- Gravitee Control plane
- IBM API Connect control plane, including the management server and analytics server
- Kong Konnect — Kong Gateway control plane
- Microsoft Azure API Management — management plane
- WSO2 API Control Plane and Management Plane

Key Characteristics

An APIM control plane implementation has the following general characteristics:

- It must expose the control plane's core capabilities, such as user management, API definition and policy definition, via RESTful APIs. These platform APIs are used to implement the admin portal and developer portal features and is also used for automation and integration purposes.
- It must maintain policy stores and distribute policy updates to API gateways.
- It must provide or facilitate certificate and secrets management for API gateway provisioning and configuration.
- It must maintain a registry or catalog for API proxy definitions and interface specifications.
- It should receive and aggregate API usage metrics and telemetry from API gateways.

This research note is restricted to the personal use of .

- It should generate API analytics and reporting for API providers and consumers, often via the admin portal and developer portals. Such API analytics can also support API monetization/chargeback capabilities.
- It should provide some basic form of built-in identity and access management when an external enterprise IAM service is not available. This built-in capability should include “superadmin” management to enable API gateway management, federation, life cycle management and control from central IAM tools.
- It should support native integration with third-party IAM tools.
- It should support integration DevOps tools and API discovery tools.

An APIM control plane implementation has the following technical characteristics:

- It should be deployed and managed independently from the data plane.
- It should offer a self-hosted option (e.g., for internal-facing APIs).
- It should offer SaaS options (e.g., for external-facing APIs).
- It should use data plane APIs to dynamically configure the data plane.
- It should publish data plane APIs (e.g., Envoy xDS) to facilitate integration with third-party API gateways.
- It should publish platform APIs for integration with third-party developer portals or IDEs.

An APIM control plane that is extended to support agentic AI gateways also has these characteristics

- It maintains a registry or catalog for AI models, MCP servers, and agents.
- It extends its mediation, observability, security, IAM and policy management capabilities to AI gateway, MCP gateway, and/or agent gateways. AI agents require much more dynamic and adaptive policy articulation and enforcement than traditional APIs.
- It offers a unified management and governance pane across traditional APIs and AI-related protocols.

This research note is restricted to the personal use of .

- It provides cost and budget management policies to optimize AI resource consumption.

Related Architecture Research

- Decision Point for Selecting the Right API Mediation Technology
- Comparing Architectures for Hybrid and Multicloud API Management
- How to Evaluate API Management Solutions
- How to Successfully Implement an API Management Platform

API Admin Portal (for API Providers)

The API admin portal, aka API manager or APIM console, is the web UI front end to the APIM control plane. It uses the platform APIs to communicate with the control plane core services as part of the platform API flow. Alternate UIs for configuring the control plane and API gateways include CLI and Kubernetes CRD.

The API admin portal serves as the policy administration point (PAP) for APIM platform operators, API product managers and API providers to administer all aspects of the API management platform.

APIM platform operators use the API admin portal and other tools to:

- Set up and configure APIM environments
- Set up multitenancy structures and role-based access control (RBAC) rules on the APIM platform to support multiple lines of business, departments or development teams
- Set up API gateway (and AI-native gateways) deployment topologies and cluster configurations

API product managers use the API admin portal to:

- Define API products
- Define API subscription plans
- Approve or deny subscription requests from API consumers

This research note is restricted to the personal use of .

- View API usage metrics and reports

API providers/developers may use the API admin portal and other tools to:

- Create API proxy definitions and API documentation
- Create and manage security, monitoring and routing policies
- Manage the life cycle of their own APIs

For APIM vendors that also provide agentic AI gateway capabilities, the admin portal should also extend its capabilities to secure, manage, and govern agentic AI resources and artifacts such as LLM services, MCP servers, and AI agents. These new capabilities can be surfaced on the same APIM admin portal to present a unified management pane or dedicated to a separate admin portal instance for better isolation and separation of duties.

Example Technologies

- APIwiz
- Axway Amplify API Management — API Manager
- Boomi Admin Portal
- Google Cloud Apigee UI
- IBM API Connect API Manager
- Kong Konnect — Gateway Manager
- Microsoft Azure portal
- Solo.io Gloo Portal
- WSO2 API Manager

Key Characteristics

An API admin portal implementation has the following general characteristics:

- It should allow the APIM platform team or administrators to configure organizations, suborganizations and environments for the APIM platform.

This research note is restricted to the personal use of .

- It should allow the APIM platform team or administrators to set up API gateway instances, cluster configurations and high availability/disaster recovery (HA/DR) options.
- It should allow the APIM platform team or administrators to define global API policies by organization, environment, application or product.
- It should allow API providers to define API proxies, mediation policies, API products and subscription plans.

An API admin portal implementation has the following technical characteristics:

- It is an integral part of the APIM platform, often with limited customization options.
- It integrates with the APIM control plane using platform APIs.

Related Architecture Research

- Decision Point for Selecting the Right API Mediation Technology
- Comparing Architectures for Hybrid and Multicloud API Management
- How to Evaluate API Management Solutions
- How to Deliver Sustainable APIs

API Developer Portal (for Consumers)

An API developer portal is a self-service website or UI designed to support developers. It covers aspects of API consumption, including:

- Access to API documentation
- API client development support, such as software development kits (SDKs) and sample codes
- API testing tools
- Self-service onboarding of API clients/consumers
- Analytics and reporting on API usage by API consumers
- Customization and branding features for APIs and API products

This research note is restricted to the personal use of .

Many organizations create separate developer portal instances — internal developer portals and external developer portals — to serve distinct purposes and target audiences:

- **Internal API developer portal:** This portal is optimized to support internal API developers as part of developer hubs.
- **External API developer portal:** This portal is the storefront that connects an organization's external API consumers to its public- or partner-facing APIs. Its primary focus is to create developer communities or to commercialize and showcase API business value.

For APIM vendors that also provide agentic AI gateway capabilities, the API developer portal should also extend its coverage to agentic AI resources such as LLM services, MCP servers, and AI agents. Start with internal API developer portal instances first, before considering publishing agentic AI resources (such as MCP and agent) in the external developer portal.

Example Technologies

Many API management products include a developer portal as part of their offering. Examples include:

- APIwiz API Marketplace
- Axway Amplify API developer portal
- Google Cloud Apigee Drupal modules and integrated portal
- IBM API Connect Developer Portal
- Kong Konnect Dev Portal
- Microsoft Azure API Management developer portal
- MuleSoft Anypoint Platform API portal
- Solo.io Gloo Portal

In addition, some stand-alone developer portal offerings include:

- Apiable

This research note is restricted to the personal use of .

- Backstage
- Bump.sh
- Microsoft Azure API Management developer portal (open-source software)
- OpsLevel
- Postman workspaces
- SwaggerHub Portal

Key Characteristics

A developer portal implementation has the following general characteristics:

- It must present to an API catalog. The API catalog may be managed by the API control plane or, in some cases, be stand-alone to support the portal user experience that must be maintained/synchronized with the runtime environment. An API catalog is the most basic requirement of an API developer portal.
- It must enable developers to register and obtain the proper credentials — such as API keys or client credentials for their consuming application — to access APIs published on the developer portal. Developers can manage their credentials, with the ability to add, modify, renew or revoke keys. This feature requires integration with the respective APIM control plane.
- It should present an MCP, Agent, or LLM services catalog alongside or as part of its API catalog with associated documentation, descriptions, and how-to-use guides.
- Developers can access usage metrics and performance analytics to help optimize their applications' API consumption. This feature requires integration with analytics and reporting capabilities, either built into the APIM platform or available from third parties.
- Developers can seek assistance from API product teams or engage with the broader developer community to exchange knowledge and share code. API providers can also use the developer portal to communicate feature updates, changes in SLAs and service contracts, and changes to relevant policies.

This research note is restricted to the personal use of .

- API client libraries, downloadable via a developer portal, allow the developer to interact with the API through a well-defined object model rather than through low-level API calls. Once they approach full coverage of all API features, such libraries are referred to as "SDKs." The easier the SDK is to use and the more functionally complete it is, the more productive a developer using it will be.

A developer portal implementation has the following technical characteristics:

- It must integrate with APIM and other components to support self-service onboarding, visibility of analytics and metrics, and maintenance of any stand-alone API catalog (if present).
- It must integrate with enterprise IAM for SSO of internal/external users via identity federation.
- It is built on a content management system (CMS), such as Drupal. Customizing/enhancing the portal may require familiarity with the underlying CMS.
- It can provide AI-assisted features to help developers design and build APIs and thereby improve developer productivity.
- It can register and catalog APIs from native API gateways or third-party gateways.

Related Architecture Research

- Leverage API Portals for Successful API Adoption
- Market Guide for API Portal Platforms

Key Architecture Principles and Patterns

Foundational principles of API management architecture include:

- **API mediation:** API gateways act as an abstraction layer between sources and consumers, decoupling them from one another. API mediation may include protocol and format transformation, content filtering and mapping, and orchestration and composition of requests and responses.
- **Dual-plane architecture:** The data plane (including API gateways) and control plane are decoupled. Control plane and data plane components can be deployed and managed independently of each other.

This research note is restricted to the personal use of .

- **Modular design:** Key API management capabilities, such as developer portals, API gateways and analytics, are independent components that can be customized, extended or replaced by customers to meet specific requirements.
- **API- and event-based communication:** API management components interact with each other using well-defined APIs and events.
- **Cloud-native design:** APIM architecture is cloud-native by design and is ready to support hybrid and multicloud deployment.
- **APIs as products:** The APIM platform embraces and supports APIs as products throughout their life cycle for all personas involved, including API providers, product managers and API consumers.
- **Customer focus:** The APIM platform enables API providers to design, develop, maintain and continuously improve APIs to meet the customers' evolving needs.

This research note is restricted to the personal use of .

Key Architecture Recommendations

- Centralize API mediation around API gateways, but avoid burdening them with complex integration tasks. Leverage complementary solutions such as integration platforms as a service (iPaaS) or other integration technology, and custom software to handle advanced integration requirements effectively.
- Leverage emerging agentic AI gateways (AI gateways, MCP gateways, and agent gateways) where appropriate to secure, govern, manage, and observe AI and agentic AI workloads across hybrid and multicloud.
- Use cloud-managed enterprise API gateways for external-facing public APIs or partner APIs to provide high availability and elasticity while minimizing operational costs. However, latency and data ingress/egress are concerns for internal APIs, especially when the API consumers and API providers are located on different clouds or on-premises networks.
- Protect and manage containerized services by using an ingress gateway to apply API-specific policies to requests from external clients. An ingress gateway helps define and secure the boundary for services deployed in a container orchestration platform.
- Choose lightweight API gateways when you need to deploy and manage your own gateways on-premises or in self-managed cloud infrastructure (containers or VMs) to minimize operational overhead. They are well-suited for governing internal APIs or domain-specific APIs that demand their own gateway instances for better isolation and customization.
- Integrate your APIM platforms or services with your enterprise IAM tools rather than using the APIM's native identity provider. This approach not only ensures long-term compliance with your enterprise security and identity federation policies but also reduces vendor lock-in.
- Leverage AI-augmented API development features in your API management platform. Start by integrating AI-driven tools into your API design, testing, and documentation workflows. Employ AI-powered testing and validation to automatically detect errors, optimize performance, and ensure compliance with best practices. Take advantage of automated documentation generation to keep API references up to date and accessible for both internal and external consumers.

Document Revision History

This research note is restricted to the personal use of .

Recommended by the Author

Some documents may not be available as part of your current Gartner subscription.

Decision Point for Selecting the Right API Mediation Technology

Comparing Architectures for Hybrid and Multicloud API Management

How to Evaluate API Management Solutions

How to Deliver Sustainable APIs

How to Successfully Implement an API Management Platform

MCP Revolutionizes Composable Agentic Applications

Innovation Insight: MCP Gateways

Market Guide for API Gateways

© 2025 Gartner, Inc. and/or its affiliates. All rights reserved. Gartner is a registered trademark of Gartner, Inc. and its affiliates. This publication may not be reproduced or distributed in any form without Gartner's prior written permission. It consists of the opinions of Gartner's research organization, which should not be construed as statements of fact. While the information contained in this publication has been obtained from sources believed to be reliable, Gartner disclaims all warranties as to the accuracy, completeness or adequacy of such information. Although Gartner research may address legal and financial issues, Gartner does not provide legal or investment advice and its research should not be construed or used as such. Your access and use of this publication are governed by Gartner's Usage Policy. Gartner prides itself on its reputation for independence and objectivity. Its research is produced independently by its research organization without input or influence from any third party. For further information, see "Guiding Principles on Independence and Objectivity." Gartner research may not be used as input into or for the training or development of generative artificial intelligence, machine learning, algorithms, software, or related technologies.

This research note is restricted to the personal use of .